

Clustering

■ Introduzione

- Definizioni
- Criteri
- Algoritmi

■ Clustering Gerarchico

■ Centroid-based Clustering

- K-means
- Fuzzy K-means
- Expectation – Maximization (Gaussian Mixture)

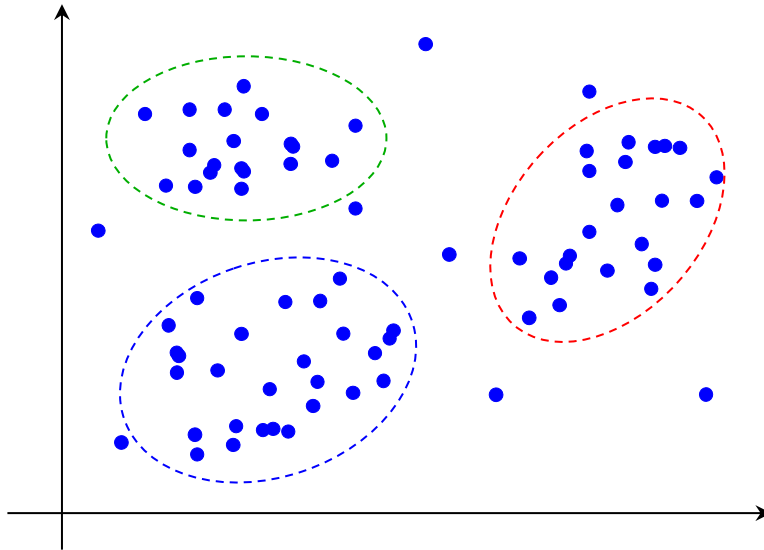
Non ci sono etichette esterne a dire cosa appartiene a cosa, quindi i gruppi emergono in base alla distribuzione dei dati nello spazio delle feature. In pratica, il modello cerca regioni dense o coese e le separa da altre regioni meno dense o distanti.

Definizioni

Il Clustering è una delle principali tecniche di apprendimento non supervisionato, cioè quei metodi in cui non si conoscono in anticipo le etichette o le classi dei dati. Lo scopo è quello di scoprire automaticamente delle strutture o dei gruppi nascosti all'interno di un insieme di dati multidimensionali.

Con il termine Clustering (in italiano «**raggruppamento**») si denota un famiglia di metodi *non supervisionati* in grado di individuare **raggruppamenti intrinseci** (cluster) di pattern nello spazio multidimensionale, e (opzionalmente) di definire in corrispondenza di tali raggruppamenti le **classi** (incognite).

Il clustering ha **applicazioni** in numerose discipline (*pattern recognition, machine learning, computer vision, data mining, data base, ...*) e pertanto ha sempre ricevuto un **notevole interesse**.



cioè la sua complessità cresce in modo esponenziale con il numero di punti e di cluster.

Il problema è **molto complesso** (NP hard): **determinare la soluzione ottima con ricerca esaustiva è possibile solo nei casi semplici** (pochi pattern).

Non esiste un algoritmo efficiente che garantisca di trovare la soluzione migliore in tutti i casi, perché bisognerebbe valutare tutte le possibili combinazioni di assegnazioni dei punti ai cluster, che sono enormemente tante già per dataset piccoli. Per questo motivo gli algoritmi di clustering più noti, utilizzano approcci euristici, capaci di fornire buone soluzioni in tempi ragionevoli, ma non necessariamente ottimali.

Quante soluzioni?

Numero di classi è noto

numero di classi

- dati n pattern, assumendo di conoscere s , il numero di soluzioni è dell'ordine di $\frac{s^n}{s!}$ (approssimazione numero di Stirling di seconda specie).

- https://it.wikipedia.org/wiki/Numeri_di_Stirling

- Esempio: per $n = 100$, $s = 5$, il numero di soluzioni $\approx 10^{67}$.

Numero di classi non è noto

- se s non è noto, il numero di soluzioni è dato dal numero di Bell, ottenibile sommando i casi precedenti per tutti i valori di s da 1 a n .

- https://it.wikipedia.org/wiki/Numeri_di_Bell

- Nel caso $s = 2$ è molto semplice dimostrare che il numero di soluzioni è $2^{n-1} - 1$.

- Esempio: dati i 4 pattern {A, B, C, D} e $s = 2$, le soluzioni di clustering sono 7:

- (A) (B,C,D)
- (B) (A,C,D)
- (C) (A,B,D)
- (D) (A,B,C)
- (A,B) (C,D)
- (A,C) (B,D)
- (A,D) (B,C)

Un criterio di clustering definisce cosa significa che una partizione dei dati è "buona" (il criterio ci serve per capire su cosa valutare la soluzione trovata). È una funzione obiettivo, o metrica di qualità, che misura quanto bene i cluster trovati rispettano certe proprietà desiderate (ad esempio, compattezza interna o separazione esterna).



Specifica il grado di ottimalità di una soluzione. Se una soluzione è "buona" o "cattiva"



Un criterio di clustering è una regola matematica che, per ogni possibile partizione valida dei dati, assegna un valore che misura quanto quella partizione è buona rispetto all'obiettivo desiderato.

Criteri di Clustering

Il criterio definisce l'obiettivo (cosa vogliamo ottimizzare). L'algoritmo è la procedura che tenta di raggiungere quell'obiettivo.

Un algoritmo di clustering è il come: è la procedura concreta per cercare (spesso approssimare) la soluzione che ottimizza quel criterio.

■ **Criteri e algoritmi di clustering sono due cose ben distinte:** i primi ~~definiscono l'obiettivo che~~ ~~descrivono cosa~~ si vuol ottenere specificando il grado di ottimalità di ogni soluzione ammissibile; i secondi, dato un criterio di clustering, forniscono una procedura algoritmica per determinare soluzioni che lo ottimizzano.

■ La maggior parte dei criteri di clustering sono definiti sulla base delle due osservazioni seguenti:

Alta similarità interna: i punti appartenenti allo stesso cluster devono essere molto simili fra loro (cioè avere distanza piccola)

1) Ridotta distanza intra-cluster, alta distanza inter-cluster

1) i pattern, all'interno dello stesso cluster, devono essere tra loro **più simili** rispetto a pattern appartenenti a cluster diversi

2) Alta densità dentro i cluster, bassa densità fuori

2) i cluster sono costituiti da **nuvole di punti** a densità relativamente elevata separate da zone dove la densità è più bassa. ➔ Bassa densità tra cluster: i cluster dovrebbero trovarsi in regioni del dominio dei dati dove la densità è alta, separati da "vuoti" (regioni a densità più bassa).

■ Tra i diversi criteri possibili:

1

cerca cluster compatti e sferici, dove ogni punto è vicino al centro del proprio gruppo.

■ **minimizzazione distanze dai centroidi:** minimizza la somma dei quadrati delle distanze dei pattern x dai centroidi (i.e. baricentri) delle classi.

Scelgo la soluzione che minimizza la distanza di ogni punto dal centroide (obiettivo: minimizzare i quadrati delle distanze)

Somma delle distanze quadratiche di ogni punto dal centroide

$$J_e = \sum_{i=1..s} \sum_{x \in C_i} \|x - \bar{x}_i\|^2$$

Somma il risultato di ogni Cluster per definire J, che deve essere minimizzato

Centroide: media dei punti nel cluster C_i

$$\bar{x}_i = \frac{1}{n_i} \sum_{x \in C_i} x$$

J_e : errore quadratico totale

dove C_i è l'i-esimo cluster, n_i il numero di pattern che contiene e $\bar{x}_i$ il suo centroide (media).

È un buon criterio per cluster a simmetria radiale (i.e., circolari), ma penalizza forme allungate o cluster innestati (i.e. un cluster a forma di anello con all'interno un altro cluster).

Il criterio dei centroidi assume che i cluster siano a forma circolare o radiale, perché la distanza Euclidea è isotropa (uguale in tutte le direzioni). Per questo motivo: funziona molto bene su cluster sferici, ben separati e di densità simile e penalizza cluster allungati, ellissoidi o annidati.

Questa formula è strettamente correlata alla minimizzazione dell'errore quadratico dal centroide, perché minimizzare la somma dei quadrati delle distanze tra tutti i punti all'interno di un cluster (varianza intra-cluster) è matematicamente equivalente a minimizzare la somma dei quadrati delle distanze di ogni punto dal centroide del cluster. Pertanto, questo criterio eredita gli stessi limiti: è ottimo per cluster sferici/radiali e penalizza le forme allungate.

Criteri di Clustering (2)

Qui si parla di minimizzazione delle distanze intra-classe, cioè si vuole che i punti appartenenti allo stesso cluster siano il più vicini possibile tra loro, senza fare esplicito riferimento ad un centroide. Invece di misurare quanto ogni punto è distante dal centro del cluster, si valuta quanto è distante dagli altri punti dello stesso cluster. L'obiettivo è ridurre la varianza interna o, più intuitivamente, la "dispersione" interna di ciascun gruppo.

2 ■ minimizzazione distanze intra-classe:

Somma pesata del numero di punti dei cluster per dispersione (Pesano di più i cluster con molti punti rispetto ad uno piccolo)

$$J_e = \sum_{i=1..s} n_i \cdot \bar{s}_i,$$

numero di pattern nel cluster C_i

$$\bar{s}_i = \frac{1}{n_i} \sum_{\mathbf{x} \in C_i} f_s(\mathbf{x}, C_i)$$

dipende da f_s

un singolo punto

dove $f_s(\mathbf{x}, C_i)$ è una misura di distanza tra $\mathbf{x}$ e il cluster cui appartiene. Ad esempio: Poi per definire f_s , faccio la media dei f_s calcolati per ogni punto del cluster

$$1. f_s(\mathbf{x}, C_i) = \frac{1}{n_i} \sum_{\mathbf{x}' \in C_i} \|\mathbf{x} - \mathbf{x}'\|^2 \quad (\text{prendo ogni punto dal cluster e calcolo la media dei quadrati delle distanze tra il punto scelto ed i restanti punti del cluster})$$

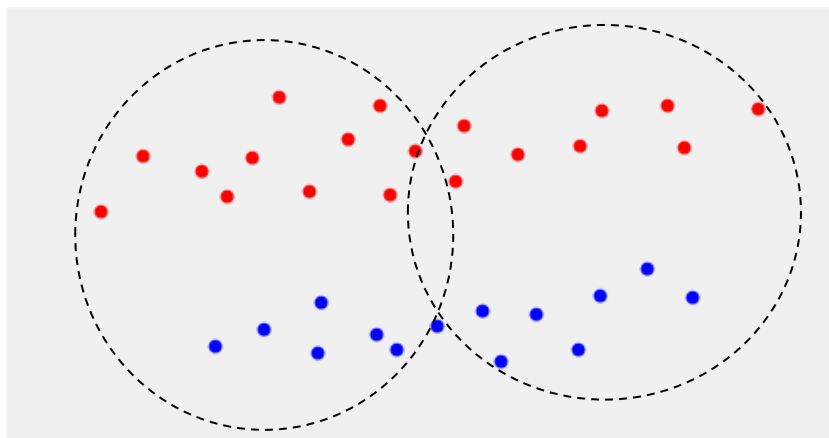
criterio simile alla minimizzazione distanze dai centroidi (slide precedente) (non esattamente come prima)

$$2. f_s(\mathbf{x}, C_i) = \min_{\substack{\mathbf{x}' \in C_i \\ \mathbf{x}' \neq \mathbf{x}}} \|\mathbf{x} - \mathbf{x}'\|^2$$

Per quel punto del cluster, f_s è il minimo delle distanze del punto scelto da tutti gli altri dello stesso cluster (mi basta cercare il punto più vicino, di quel cluster, al punto scelto per il calcolo)

non penalizza cluster allungati, infatti affinché $f_s(\mathbf{x}, C_i)$ assuma valore ridotto, non è necessario che tutti i (o gran parte dei) pattern di C_i siano vicini a $\mathbf{x}$, ma è sufficiente un vicino. Nell'esempio sotto, per $s = 2$, (1) favorirebbe l'aggregazione come da cerchi tratteggiati, mentre (2) come da colori (rosso e blu).

I cluster possono "allungarsi" perché l'ottimalità è garantita dalla vicinanza locale dei punti, non dalla loro compattezza globale attorno a un centro.



Algoritmi di Clustering

■ Le principali famiglie di algoritmi sono:

- **Clustering gerarchico:** attraverso operazioni tipicamente «bottom-up», che aggregano pattern in base a una misura di distanza, si organizzano i dati in struttura ad albero (dendrogramma).

- **Clustering basato su centroidi:** attraverso euristici (iterativi) si individuano i cluster cercando di minimizzare la distanza dei pattern dai centroidi dei cluster cui appartengono:

- K-means
- Fuzzy K-means
- Expectation – Maximization (Gaussian Mixture)

- **Clustering basato sulla densità:** i cluster individuati sono regioni connesse in aree ad elevata densità. L'approccio più noto è DBSCAN (<https://en.wikipedia.org/wiki/DBSCAN>)

significa che l'algoritmo sta raggruppando insieme tutti i punti che formano una massa continua e compatta, separandoli dalle zone dove i punti sono troppo rari.

■ Distinguiamo inoltre:

- **Clustering hard (esclusivo):** un pattern è assegnato (in modo esclusivo) a un solo cluster. Il risultato è un partizionamento discreto.



Tutti i punti appartengono a tutti i cluster con probabilità associata (non implica che non si può fare assoc. esclusiva)

- **Clustering soft (fuzzy):** i pattern appartengono ai diversi cluster con un certo grado di appartenenza (es. tra 0 e 1).

È più efficace nel gestire pattern vicino al bordo di due o più clusters e outliers. L'assegnazione può diventare esclusiva scegliendo, per ogni pattern, il cluster verso cui il grado di appartenenza è massimo.

Il Clustering Gerarchico è di tipo Hard, ma non abbiamo il vincolo di associare tutti i pattern ad un cluster (alcuni pattern in cluster piccolissimi). Questo rende il clustering gerarchico, in un certo senso, tollerante verso i dati che non si adattano bene (outliers), a patto che la soglia di taglio sia opportunamente definita per isolarli.

L'algoritmo rimane lo stesso, ma cambia il calcolo delle distanze (cioè cambia i risultati dell'algoritmo)

Clustering Gerarchico

Inizio: Ogni punto dati è un cluster autonomo. Se abbiamo N punti, abbiamo N cluster. Ad ogni passo, i due cluster (o pattern) che sono più simili vengono fusi per formarne uno nuovo. Il processo continua finché non si raggiunge un unico cluster che contiene tutti i dati, o finché la distanza tra i cluster più vicini non supera una soglia predefinita.

Simile al modo di eseguire **classificazione in tassonomia biologica**:

- gli insetti sono **gerarchicamente** classificati **specializzandone** le specie a partire da famiglie molto **ampie** fino a famiglie più **ridotte**.

L'albero può non essere binario

Gli algoritmi sono generalmente **bottom-up** (agglomerativi):

- si parte cercando di **aggregare** singoli elementi e ad ogni passo (livello) si **aggregano** (*pattern a pattern, pattern a cluster, o cluster a cluster*) gli elementi tra loro più simili (i.e. **meno distanti**) rispetto a una **soglia** (che dipende dal livello).

Quando siamo nella prima fase, consideriamo i pattern come Cluster Autonomi (cioè Cluster contenenti solo quel pattern)

- Le principali differenze implementative dipendono dalla definizione utilizzata per calcolare le distanze tra cluster e cluster: (Questa definizione è cruciale perché determina quali cluster verranno fusi ad ogni passo.)

La distanza tra due cluster è la distanza minima tra una qualunque coppia di punti, uno per ciascun cluster.

- **Single link:** distanza minima tra due pattern dei cluster

La distanza tra due cluster è la distanza media tra tutte le coppie di punti dei due cluster.

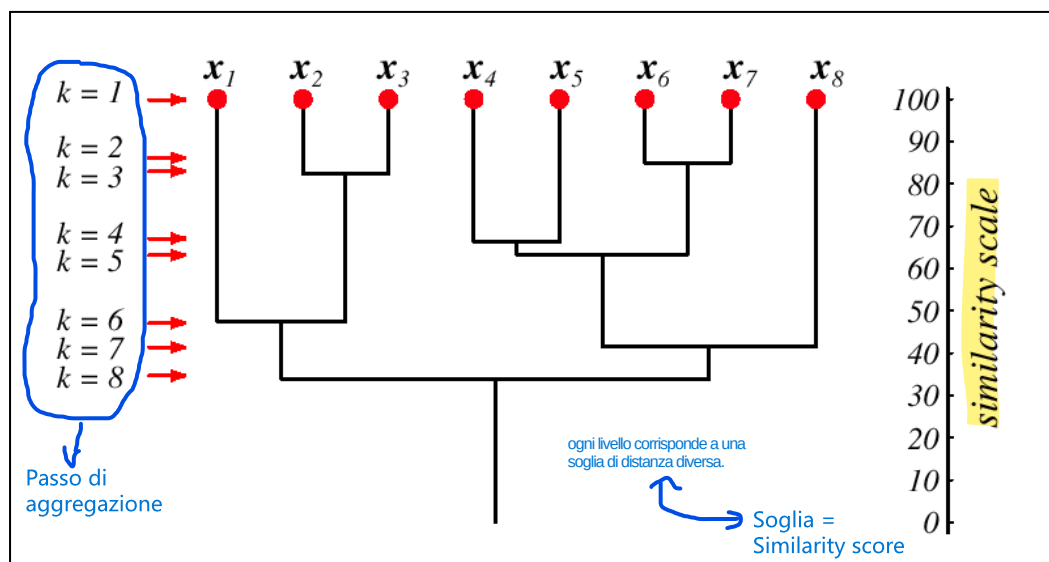
- **Average link:** distanza media tra i pattern dei due cluster

La distanza tra due cluster è la distanza massima tra una qualunque coppia di punti, uno per ciascun cluster.

- **Complete link:** distanza massima tra due pattern dei cluster

Tende a formare cluster sferici. Sensibile agli outlier

Tende a formare cluster allungati



Rappresenta un buon compromesso tra il Single Link e il Complete Link: tende a essere meno sensibile agli outlier e meno incline all'allungamento.

Poiché l'ottimizzazione è iterativa e locale, una cattiva scelta dei centroidi iniziali può portare l'algoritmo a convergere a un massimo locale (un risultato sub-ottimale) invece che alla soluzione globale migliore. Per mitigare questo, si esegue spesso l'algoritmo più volte con inizializzazioni diverse (es. K-means++).

K-means

È popolare grazie alla sua semplicità computazionale e di implementazione (metodo euristico).

K-means (o C-means) è un metodo computazionalmente molto semplice e altrettanto semplice da implementare: per questo motivo è spesso la prima scelta per risolvere problemi di clustering.

L'algoritmo cerca di rendere i cluster il più compatti possibile.

- Minimizza «implicitamente» le distanze dai centroidi.
- Richiede in input il numero di cluster (s) e una soluzione iniziale. Produce buoni risultati a patto di fornire una ragionevole soluzione iniziale e un numero adeguato di classi.
- Il tipo di ottimizzazione è iterativa e locale; pertanto il metodo può convergere a ^{ad una} massimi locali ^{sub-ottimale} della soluzione. La convergenza si ottiene solitamente in pochi passi: < 10).
- Identifica cluster iper-sferici nel caso in cui venga utilizzata la distanza euclidea come misura di distanza tra i pattern o cluster iper-ellissoidali nel caso di distanza di Mahalanobis.
- Nella sua versione base, l'algoritmo può essere così descritto:



Inizializza n, s

n : numero di pattern
 s : numero di cluster

La versione base usa
distanza euclidea

Scegli casualmente s pattern da utilizzare come centroidi iniziali

do { assegna ogni pattern al cluster ^{j-esimo} per cui è minima la distanza dal centroide. ^{del cluster j-esimo}

2 calcola i centroidi dei cluster come media dei rispettivi pattern

} while (i cluster sono stati modificati & $iteration < max$)

Assegno i pattern ai cluster e ri-calcolo i centroidi e ri-assegno i pattern ai cluster e così via

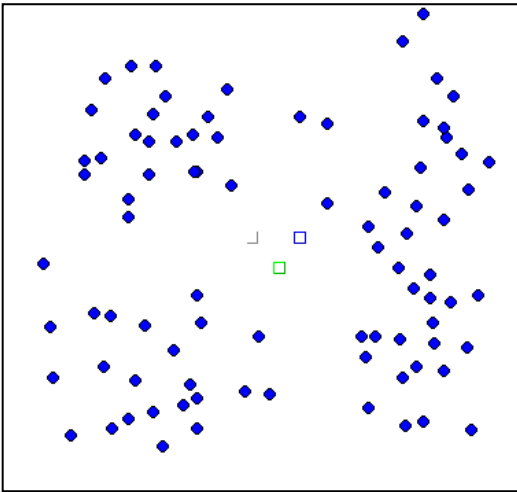
I cluster sono modificati iterativamente a seguito del ricalcolo del loro centroide. L'algoritmo termina (converge) quando i centroidi sono stabili e quindi le partizioni non cambiano.

oppure termina quando raggiungo il numero di iterazioni fissate come massimo (di solito, viene raggiunta questa condizione quando un pattern, ad ogni iterazione, senza guardia, continuerebbe a spostarsi di cluster all'infinito)

K-means: esempio ($n = 86, s = 3$)

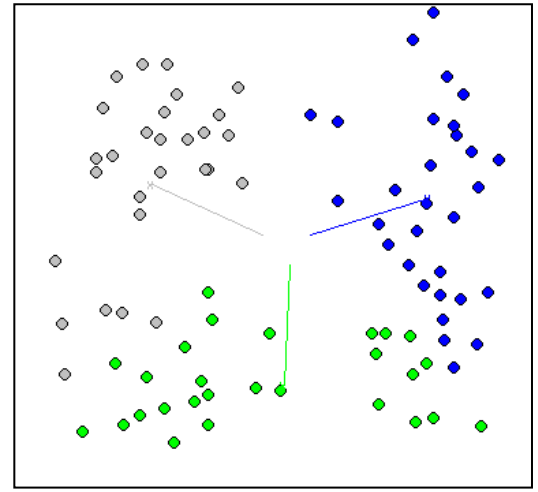
Vengono scelti a caso i tre centroidi come soluzione iniziale (in questo caso, generati a caso i punti, non presi dai punti da assegnare ai cluster)

soluzione iniziale



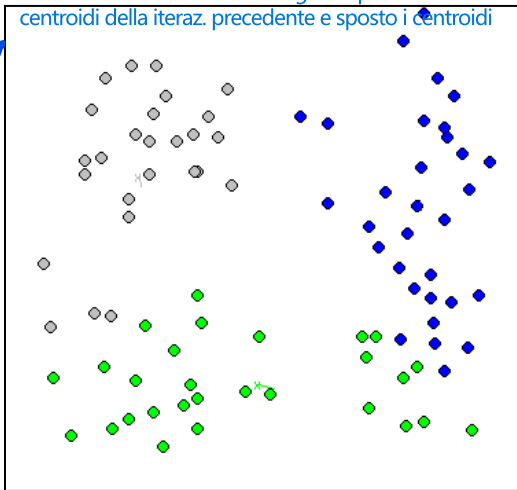
Assegno i pattern ai cluster rispetto alla posizione dei centroidi della soluzione iniziale, poi sposto i 3 centroidi dei 3 cluster nella posizione segnata con la 'x'

iterazione 1

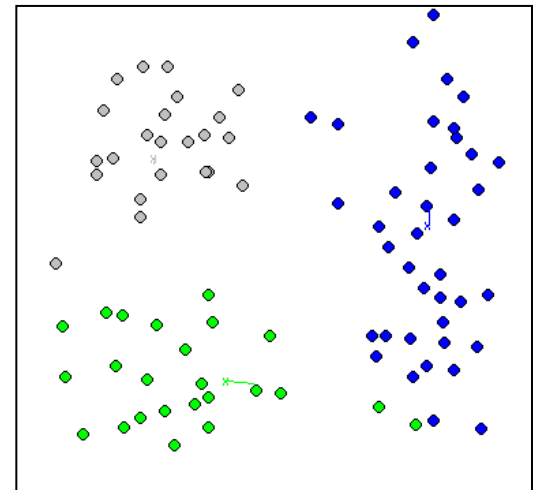


Stessa cosa della iteraz. 1: assegno rispetto ai centroidi della iteraz. precedente e sposto i centroidi

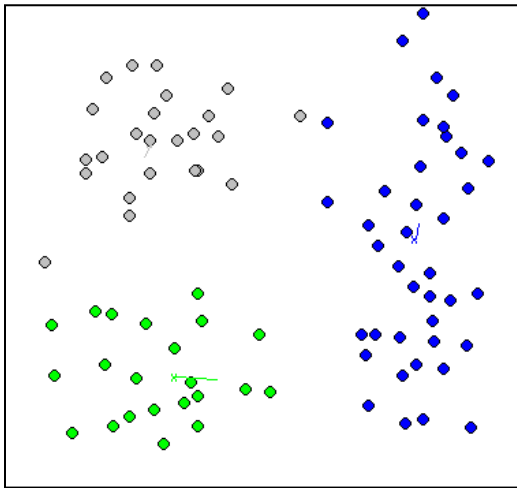
iterazione 2



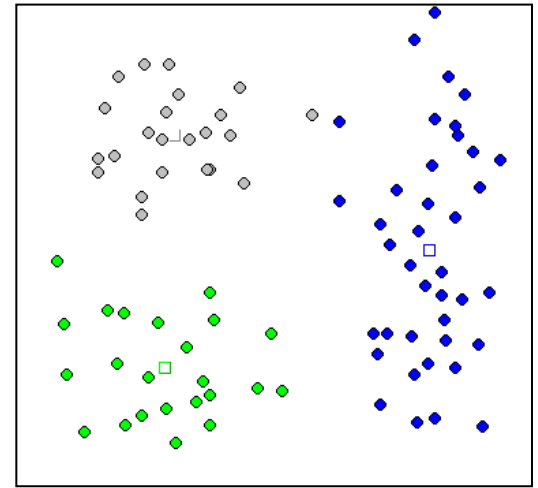
iterazione 4



iterazione 6



iter. 7 (convergenza)

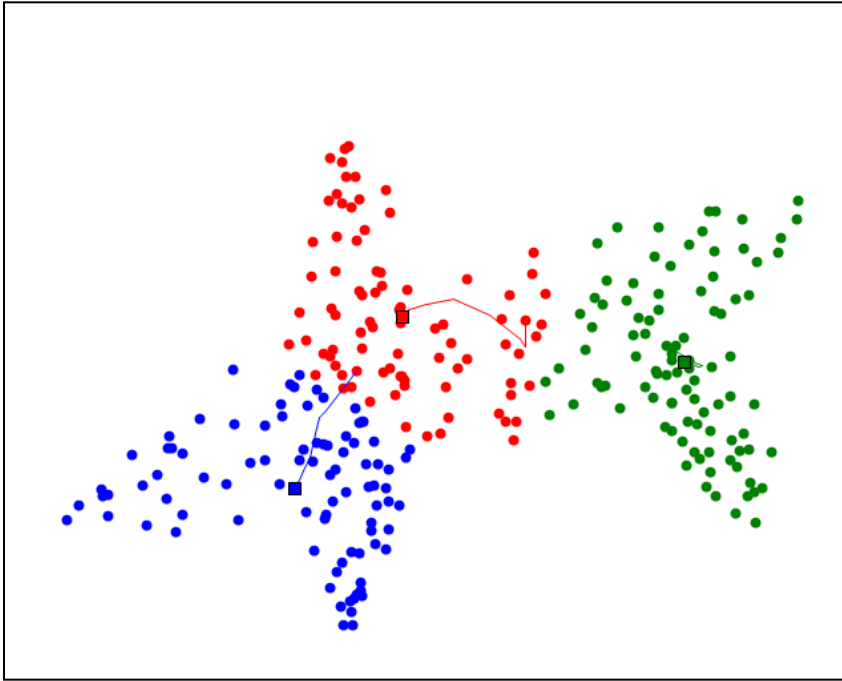


Gli assegnamenti ed il ricalcolo dei centroidi cambiano la forma e la posizione dei cluster. Si fa ciò fino a convergenza o all'arrivo della iterazione massima

Per risolvere, si potrebbe usare la distanza di Mahalanobis. Usare la Distanza di Mahalanobis in un algoritmo basato su centroidi come il K-means è un modo per permettere all'algoritmo di identificare cluster iper-ellissoidali (più allungati, inclinati o di forma non sferica).

K-means: limitazioni

La limitazione fondamentale del K-means, specialmente quando si usa la distanza euclidea, è la sua assunzione intrinseca che i cluster siano iper-sferici.

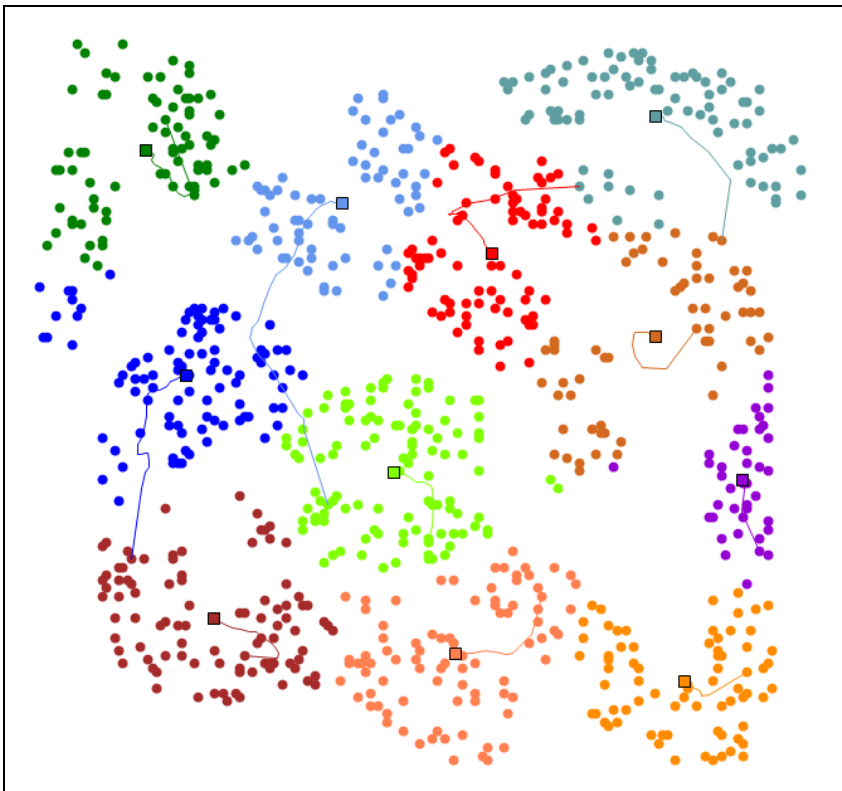


3 classi – 8 iterazioni

Minimizzando le distanze dai centroidi, K-means non è in grado di identificare cluster dalla forma non sferica.

Riesce ad identificare solo cluster sferici, se si usa la distanza euclidea

Se i dati formano anelli, mezzelune, o qualsiasi altra forma arbitraria complessa, il K-means non è l'algoritmo adatto.



11 classi, 31 iteraz.

anche in questo caso non tutti i cluster hanno forma sferica ...

K-means: soluzione iniziale e validazione

Diverse **varianti** sono state proposte per **generare buone soluzioni iniziali** e di **determinare il numero di classi** (clustering validation).

■ Per minimizzare il rischio di **convergenza verso minimi locali**, l'algoritmo può essere eseguito più volte a partire da soluzioni iniziali diverse:

A ■ **random**

La soluzione più semplice è scegliere casualmente K punti dati (o K posizioni) come centroidi iniziali. È la base, ma è molto sensibile alle fluttuazioni casuali.

B ■ **prodotte da un (diverso) euristico**

utilizzano l'ottimizzazione globale per cercare combinazioni di centroidi iniziali che minimizzino la funzione obiettivo. Sono potenti ma computazionalmente costosi.

C ■ **o magari da un metodo evoluzionistico (algoritmo genetico).**

La pratica più comune è eseguire l'algoritmo più volte (es. 10, 50 o 100 volte) con diverse inizializzazioni e scegliere il risultato (il set di centroidi e cluster) che ha prodotto il minimo valore finale di J.

② **Quanti cluster ?**

Aumentando il numero di cluster s, diminuisce J. Quindi, possiamo usare J per comparare solo le soluzioni con stesso s, non con s diverso

■ Le tecniche di **validazione** tendono a valutare a posteriori la bontà delle soluzioni prodotte per diversi valori di **s**, e a sceglierne una sulla base di un **criterio di validazione** che tenga conto sia della bontà della soluzione sia della sua complessità:

■ **Problema:** considerando il criterio di «minimizzazione distanze dai centroidi» e lanciando K-means con diversi valori di **s**, è molto più facile ottenere valori ridotti di J_e per valori elevati di **s**. Quanto vale J_e per **s = n** (ogni cluster un solo pattern) ? Non possiamo usare solo la minimizzazione di J_e per scegliere s, altrimenti la scelta sarebbe sempre $s=n$.

■ **Possibile soluzione:** penalizzare le soluzioni con molti cluster; ad esempio:

$$J_e^* = J_e + \text{penalty} \cdot s$$

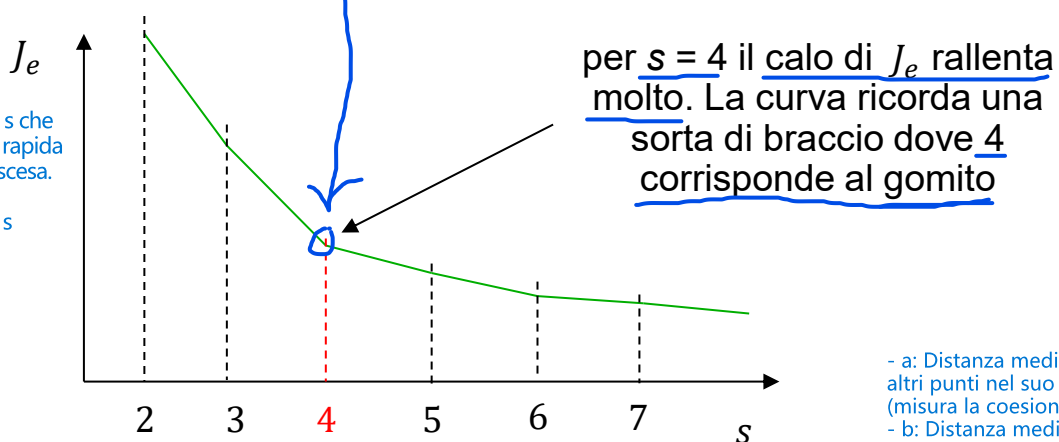
Di fatto però il problema si ribalta nella scelta di **penalty**

L'idea è trovare un equilibrio: un basso J_e (buona compattezza) senza un s eccessivo. Tuttavia, come notato, questo problema si "ribalta" nella scelta di un valore sensato per il termine di penalità.

K-means: validazione (2)

Discrezionalità nella scelta del 'gomito' nel grafico

Discontinuità nel grafico: un modo più efficace per determinare il numero ottimale di cluster è quello di cercare punti di inflessione (anche detti *elbow*) nel valore di J_e al variare di s . Infatti, se i pattern formano raggruppamenti evidenti e ben identificabili, scegliendo il corretto valore di s , dovremmo riscontrare una discontinuità nel grafico di J_e al variare di s .



Scegliere il valore s che fa da 'gomito' tra rapida discesa e lenta discesa. Poi prendere il J_e minimo con quel s

- a : Distanza media tra i e tutti gli altri punti nel suo stesso cluster (misura la coesione).
- b : Distanza media tra i e i punti del cluster vicino (il cluster ad esso più vicino) (misura la separazione).

Il Silhouette Score è un criterio più rigoroso che misura quanto bene ogni punto è raggruppato all'interno del proprio cluster rispetto agli altri cluster.

Silhouette score: un criterio più oggettivo (che funziona per cluster sferici come quelli prodotti da k-means) è quello di calcolare il silhouette score come media dei *silhouette coefficient* di ciascun pattern e scegliere s che lo massimizza.

$$\text{Silhouette coefficient}_i = \frac{(b - a)}{\max(a, b)}$$

($b-a$) viene normalizzato con il massimo tra i due

dove a è la distanza media intra cluster del pattern e b la distanza tra il pattern e i pattern del cluster ad esso più vicino (escluso quello di appartenenza). È presente in scikit-learn la funzione `silhouette_score()` che calcola questa metrica. Risulta però computazionalmente pesante per grandi dataset.

Asse Verticale: Ogni riga rappresenta un singolo pattern (punto dati).

Asse Orizzontale: L'altezza della barra rappresenta il Coefficiente di Silhouette (s_i) di quel pattern, che va da -1 a +1.

Il **silhouette diagram** (vedi [A. Géron]) è una rappresentazione grafica degli score utile per la validation.

Si esegue l'algoritmo di clustering (K-means) per vari valori di s . Per ogni risultato si calcola lo Score di Silhouette medio. Si sceglie il valore di s che produce lo Score di Silhouette medio più alto, in quanto indica la partizione in cluster che è contemporaneamente più compatta (bassa a) e più separata (alta b).

Mentre il K-means standard (Hard) assegna rigidamente ogni punto ad un unico cluster, il Fuzzy K-means consente a un pattern x_j di appartenere a tutti i cluster, ma con un diverso grado di appartenenza o "probabilità" $P(C_i | x_j, \Theta)$.

Fuzzy K-means

La distanza euclidea tra due punti (o vettori) (x) e (y) è definita come la norma 2 della loro differenza:
 $d(x,y) = \text{norma } 2 \text{ di } x-y$

La variante **Fuzzy** del K-means consente a un pattern di appartenere con un certo grado di probabilità a diverse classi. Il criterio di ottimizzazione in questo caso è:

è il quadrato della distanza Euclidea tra il pattern x_j e il centroide θ_i .

$P(C_i | x_j, \theta_i)^m$ è il grado di appartenenza del pattern x_j al cluster C_i , elevato alla potenza m .

$$J_{fuz} = \sum_{i=1..s} \sum_{j=1..n} [P(C_i | x_j, \Theta)]^m \cdot \|x_j - \theta_i\|^2$$

dove $P(C_i | x_j, \Theta)$ è la probabilità che dato il pattern x_j e l'insieme $\Theta = \{\theta_1 \dots \theta_s\}$ di centroidi che definiscono la soluzione attuale, il cluster di appartenenza sia C_i .

I **centrodi** invece che come semplice media dei pattern si calcolano come media pesata rispetto alle probabilità di appartenenza:

I pattern che hanno un alto grado di appartenenza (vicini a 1) al cluster C_i avranno un peso maggiore nel definire la posizione del nuovo centroide θ_i . Un pattern di confine (con basso P) contribuirà meno.

$$\theta_i = \frac{\sum_{j=1..n} P(C_i | x_j, \Theta) \cdot x_j}{\sum_{j=1..n} P(C_i | x_j, \Theta)}$$

Eq. (1)

devo far influire poco nel calcolo del centroide i data point che hanno una bassa probabilità di appartenenza a quel cluster

Le **probabilità di appartenenza** si calcolano come:

L'appartenenza di x_j al cluster C_i è inversamente proporzionale alla sua distanza dal centroide θ_i . Più x_j è vicino a θ_i (distanza $\|x_j - \theta_i\|^2$ piccola), più il grado $P(C_i | x_j, \theta_i)$ sarà alto.

Più aumenta la distanza del punto x_j dal centroide, più la probabilità di appartenenza diminuisce

$$P(C_i | x_j, \Theta) = \frac{1}{\sum_{k=1..s} \left(\frac{\|x_j - \theta_i\|^2}{\|x_j - \theta_k\|^2} \right)^{\frac{2}{m-1}}}$$

Eq. (2)

La formula penalizza l'appartenenza a un cluster C_i ogni volta che un altro cluster C_k è più vicino, garantendo che l'appartenenza sia concentrata sui centroidi più prossimi.

$m > 1$ è un parametro che definisce quanto l'appartenenza ai diversi cluster debba essere **sfumata**. Valore tipico **$m = 2$** .

Il parametro m è cruciale:

- m : Definisce la sfumatura. Controlla la "sfocatura" o la "fuzzy-ness" dell'assegnazione.

> m vicino a 1: L'algoritmo si comporta in modo molto simile al K-means Hard. Il grado di appartenenza tende a 0 o 1.

La funzione si comporta come un filtro duro che assegna quasi tutto il peso al centroide più vicino, ignorando gli altri.

> m alto ($m=5$ o superiore): il clustering diventa molto sfumato. Le probabilità di appartenenza di un punto sono distribuite in modo più uniforme tra i cluster, anche se il punto è relativamente vicino a un solo centroide. I centroidi diventano meno sensibili ai punti vicini.

> m verso l'infinito: Il grado di appartenenza tende a essere uguale per tutti i cluster ($1/s$), indipendentemente dalla distanza. Il clustering diventa inutile.

significa che l'appartenenza a un cluster non è 0 o 1, ma una quantità continua che riflette quanto un punto è vicino ai diversi cluster; il parametro m controlla quanto questa appartenenza è distribuita o concentrata.

In sintesi, m bilancia la rigidità (come K-means) con la tolleranza alla sovrapposizione (il "fuzzy" del metodo).

Fuzzy K-means (2)

è un'estensione naturale del K-means standard, mantenendo la stessa logica iterativa a due passi.

Il pseudocodice dell'algoritmo è molto simile al K-means:

Il ciclo si alterna tra l'aggiornamento dei centroidi e l'aggiornamento dei gradi di appartenenza:

Inizializza n, s

Inizializza casualmente $P(C_i | \mathbf{x}_j, \Theta)$, $i = 1 \dots s, j = 1 \dots n$ Appartenenza per ogni cluster e per ogni dato A differenza del K-means standard, si inizializzano i gradi di appartenenza

do { 1 calcola i centroidi $\Theta = \{\theta_1 \dots \theta_s\}$ // Eq. (1) Si calcolano i centroidi come media pesata

2 le probabilità calcola i gradi di appartenenza $P(C_i | \mathbf{x}_j, \Theta)$, $i = 1 \dots s, j = 1 \dots n$ // Eq. (2) Si ricalcolano i gradi di appartenenza

} while (riduzione J_{fuz} significativa & iteration < max)

Condizione di stop: La funzione obiettivo J_{fuz} non si riduce in modo significativo oppure si raggiunge l'iteraz. massima

Rispetto al K-means, questa variante fornisce a volte una convergenza più robusta verso la soluzione finale. D'altro canto uno svantaggio è legato al fatto che l'appartenenza di un pattern a un cluster dipende implicitamente dal numero di cluster, e se questo è specificato in modo incorretto si possono ottenere cattive soluzioni.

Il Fuzzy K-means è spesso considerato più robusto rispetto alla sua controparte Hard.

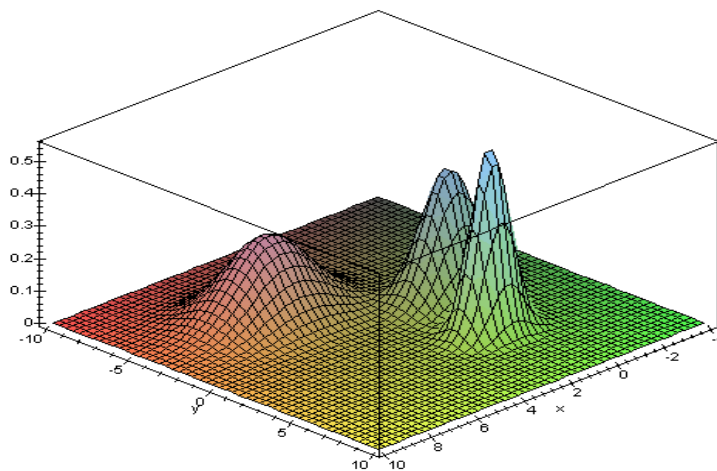
- Motivazione: Nel K-means standard, lo spostamento di un singolo punto di confine da un cluster all'altro può causare un drastico cambiamento nella posizione del centroide e innescare grandi oscillazioni nelle iterazioni successive. Nel Fuzzy K-means, lo stesso punto di confine contribuisce a entrambi i centroidi con un peso intermedio (ad esempio 0.5 e 0.5), rendendo il movimento dei centroidi più graduale e stabile e portando a una convergenza più liscia.

Questo è il principale difetto di tutti i metodi basati su centroidi: l'efficacia dipende dalla specifica corretta del numero di cluster s . Problema: Se si sceglie un valore di s troppo alto o troppo basso, le probabilità di appartenenza saranno distorte, portando a partizioni che non riflettono la vera struttura dei dati e quindi a cattive soluzioni.

Expectation – Maximization (EM)

con Gaussian Mixture

Si ipotizza che i pattern siano stati generati da un *mix di distribuzioni* ("mixture" in inglese): ogni classe ha generato dati in accordo con una specifica distribuzione, ma al termine della generazione i pattern appaiono come prodotti da un'unica distribuzione multi-modale.



Immagina che ci siano s diverse "macchine" (i cluster) che generano dati. Ogni macchina i -esima genera dati secondo la sua specifica distribuzione di probabilità $p_i(x | \theta_i)$ e viene scelta con una certa probabilità α_i (il peso del cluster). Quando guardiamo i dati finali (l'insieme di tutti i pattern), questi appaiono come prodotti da una singola distribuzione multi-modale (cioè con più "gobbe", come mostrato nel grafico). Il nostro compito è svelare le singole "gobbe" nascoste.

Obiettivo del clustering con EM è risalire (a partire dai pattern del training set) ai parametri delle singole distribuzioni che li hanno generati. A tal fine, si ipotizza *nota* la forma delle distribuzioni e si assume, per semplicità, che esse siano tutte dello stesso tipo. Il caso più frequente è quello di *mix* di s distribuzioni *multinormali* (gaussiane), di cui si vogliono *stimare* i parametri di definizione (s vettori medi + s matrici di covarianza + s coefficienti α):

$$p(\mathbf{x}|\Theta) = \sum_{i=1..s} \alpha_i \cdot p_i(\mathbf{x}|\theta_i)$$

$$\theta_i = \{\mu_i, \Sigma_i\}$$

dove $\Theta = \{\alpha_1, \alpha_2 \dots \alpha_s, \theta_1, \theta_2 \dots \theta_s\}$ è il vettore di parametri che definisce il mix di distribuzioni, $p_i(\cdot | \theta_i)$ è una multinormale con parametri θ_i

- Vettore medio μ_i (la posizione del centroide).
- Matrice di Covarianza Σ_i (la forma e l'orientamento del cluster, permettendo forme ellissoidali, superando il limite sferico del K-means).
- Peso α_i (È una probabilità: indica quanto spesso, in media, viene scelto quel cluster nella fase di generazione. Tutti gli α_i sono positivi e sommano a 1).

EM: Idea

L'EM è un potente algoritmo iterativo che mira a trovare la stima dei parametri Theta che rende massima la probabilità di aver osservato i dati X

La stima dei parametri avviene secondo il criterio generale del maximum likelihood (in italiano “massima verosimiglianza”).

↪ è un metodo per trovare i parametri di un modello che meglio si adattano ai dati che abbiamo osservato.

In generale il likelihood $\mathcal{L}$ dei parametri Θ dati i pattern $\mathcal{X}$ corrisponde alla probabilità (densità di probabilità nel continuo) di aver ottenuto i pattern, dati i parametri (inversione):

$$\mathcal{L}(\Theta|\mathcal{X}) = p(\mathcal{X}|\Theta)$$

Per semplicità, al posto del likelihood, si massimizza il suo logaritmo. Considerando i pattern indipendenti, la probabilità di avere ottenuto i pattern del training set è il prodotto delle ^{densità di} probabilità delle singole generazioni:

$$\log p(\mathcal{X}|\Theta) = \log \prod_{j=1 \dots n} p(\mathbf{x}_j|\Theta) = \sum_{j=1 \dots n} \log \left(\sum_{i=1 \dots s} \alpha_i \cdot p_i(\mathbf{x}_j|\theta_i) \right)$$

Purtroppo questa massimizzazione è molto difficile a causa della sommatoria dentro al logaritmo. Per eliminare la sommatoria, ci servirebbe sapere ogni pattern $\mathbf{x}_j$ del training set da quale componente $p_i(\cdot)$ della mixture è stato generato.

↪ Se conoscessimo la variabile latente Y, la sommatoria scomparirebbe e potremmo trattare ogni pattern come proveniente da una singola distribuzione Gaussiana, rendendo la massimizzazione gestibile.

A tal fine si utilizza EM, che è un approccio iterativo nato per il calcolo del maximum likelihood nel caso in cui i dati a disposizione $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2 \dots \mathbf{x}_n\}$ siano incompleti per la mancanza di alcuni valori $\mathcal{Y} = \{\mathbf{y}_1, \mathbf{y}_2 \dots \mathbf{y}_n\}$. Ogni pattern completo $\mathbf{z}_j = [\mathbf{x}_j, \mathbf{y}_j]$, $j = 1 \dots n$ è costituito da due parti di cui solo la prima è nota.

Nel caso di interesse (derivare i parametri di una gaussian mixture) i dati sono completi ma le potenzialità di EM sono utilizzate per rendere trattabile la massimizzazione: sfruttiamo i valori $\mathbf{y}$ come indicatori delle componenti ignote che hanno generato i singoli pattern.

Dati mancanti y: È l'informazione su "quale campana gaussiana ha generato questo punto". Questa informazione è ignota.

EM: nel caso generale

SALTA
QUESTA
PAGINA

Obiettivo di EM è dunque la massimizzazione del **log-likelihood** dei **dati completi** o **log-likelihood-completo**:

$$\log \mathcal{L}(\Theta|Z) = \log \mathcal{L}(\Theta|\mathcal{X}, \mathcal{Y}) = p(\mathcal{X}, \mathcal{Y}|\Theta)$$

A tal fine vengono eseguiti **iterativamente** (fino a convergenza) due passi, **Expectation** e **Maximization**:

- **Expectation**: viene calcolato il valore atteso $E(\cdot)$ del log-likelihood-completo, **dato** il **training set** $\mathcal{X}$ e una **stima dei parametri** Θ^g ottenuti all'iterazione precedente:

$$Q(\Theta|\Theta^g) = E(\log p(\mathcal{X}, \mathcal{Y}|\Theta) | \mathcal{X}, \Theta^g)$$

Il valore atteso (**media**) è calcolato rispetto alla variabile aleatoria $\mathcal{Y}$, governata dalla distribuzione $f(\mathcal{Y}|\mathcal{X}, \Theta^g)$:

$$E(\log p(\mathcal{X}, \mathcal{Y}|\Theta) | \mathcal{X}, \Theta^g) = \int_{\mathcal{Y} \in \Psi} \log p(\mathcal{X}, \mathcal{Y}|\Theta) \cdot f(\mathcal{Y}|\mathcal{X}, \Theta^g) d\mathcal{Y}$$

- **Maximization**: determina il valore dei **parametri** che massimizzano il valore atteso calcolato al passo di **Expectation**:

$$\Theta^{g+1} = \underset{\Theta}{\operatorname{argmax}} Q(\Theta|\Theta^g)$$

EM: nel caso specifico

Sfruttando le peculiarità di EM, si **assume** l'esistenza di **una componente nascosta** y_j (per ogni pattern) che indica quale delle s distribuzioni gaussiane ha generato il pattern $\mathbf{x}_j$.

Il **log-likelihood-completo** assume ora la forma:

$$\log \mathcal{L}(\Theta | \mathcal{X}, \mathbf{y}) = \sum_{j=1 \dots n} \log \left(\alpha_{y_j} \cdot p_{y_j}(\mathbf{x}_j | \boldsymbol{\theta}_{y_j}) \right)$$

sebbene gli y_j non siano noti, **una loro stima** (o meglio una stima della **loro distribuzione**) può essere derivata (**teorema di Bayes**) dai parametri Θ^g disponibili all'iterazione (g) corrente.

Per un singolo y_j (e corrispondente $\mathbf{x}_j$):

$$P(y_j = k | \mathbf{x}_j, \Theta^g) = P(k | \mathbf{x}_j, \Theta^g) = \frac{\alpha_k^g \cdot p_k(\mathbf{x}_j | \boldsymbol{\theta}_k^g)}{\sum_{i=1 \dots s} \alpha_i^g \cdot p_i(\mathbf{x}_j | \boldsymbol{\theta}_i^g)} \quad \text{Eq. (1)}$$

Per un generico vettore $\mathbf{y}$ di osservazioni:

$$P(\mathbf{y} | \mathcal{X}, \Theta^g) = \prod_{j=1 \dots n} P(y_j | \mathbf{x}_j, \Theta^g)$$

Il valore atteso Q della slide precedente può essere scritto come:

$$\begin{aligned} Q(\Theta | \Theta^g) &= \sum_{\mathbf{y} \in \Psi} \log \mathcal{L}(\Theta | \mathcal{X}, \mathbf{y}) \cdot P(\mathbf{y} | \mathcal{X}, \Theta^g) = \\ &= \sum_{\mathbf{y} \in \Psi} \left(\sum_{j=1 \dots n} \log \left(\alpha_{y_j} \cdot p_{y_j}(\mathbf{x}_j | \boldsymbol{\theta}_{y_j}) \right) \cdot \prod_{j=1 \dots n} P(y_j | \mathbf{x}_j, \Theta^g) \right) \end{aligned}$$

Questo denominatore indica il Peso Totale Effettivo (o Atteso) del cluster C_k all'interno del dataset. Non è un numero intero di punti, ma la loro somma probabilistica: Non è un Conteggio Hard: non possiamo semplicemente contare i punti. È una Somma di Pesi: Il denominatore somma tutti questi pesi (da 0 a 1) per ottenere il "numero equivalente" di punti che contribuiscono effettivamente a definire C_k (Ad esempio, dieci punti con responsabilità 0.1 sommano a 1.0\$, equivalenti a un singolo punto intero).

I valori sono considerati "attesi" perché, in questo contesto EM, stiamo lavorando con probabilità e variabili latenti, non con assegnazioni certe.

Viene messo al denominatore per agire da fattore di normalizzazione nel contesto del Maximum Likelihood, garantendo che i parametri siano calcolati come una media attesa probabilistica e non una semplice somma.

EM: formule incremental

A seguito di alcuni passaggi (non banali) [1] durante i quali l'ottimizzazione è eseguita eguagliando a 0 le derivate parziali rispetto ai parametri incogniti si ottengono le seguenti formule per l'aggiornamento incrementale dei parametri:

$$P(C_k | \mathbf{x}_j, \theta^g) = \frac{\alpha_k^g \cdot p(\mathbf{x}_j | \theta_k^g)}{\sum_{i=1 \dots s} \alpha_i^g \cdot p(\mathbf{x}_j | \theta_i^g)}$$

Eq. (1)

peso del cluster α_k moltiplicato per la densità di probabilità che $\mathbf{x}_j$ sia stato generato dalla Gaussiana k .

È la somma dei numeratori per tutti i cluster (s), che serve come fattore di normalizzazione per assicurare che la somma delle probabilità di $\mathbf{x}_j$ su tutti i cluster sia 1.

$$\alpha_k^{g+1} = \frac{1}{n} \sum_{j=1 \dots n} P(C_k | \mathbf{x}_j, \theta_k^g)$$

Eq. (2)

Il nuovo peso del cluster α_k è dato dalla media dei gradi di appartenenza di tutti i pattern al cluster C_k . Se un cluster ha una forte responsabilità su molti pattern, il suo peso aumenta.

$$\mu_k^{g+1} = \frac{\sum_{j=1 \dots n} \overbrace{P(C_k | \mathbf{x}_j, \theta_k^g)}^{\text{Peso}} \cdot \mathbf{x}_j}{\sum_{j=1 \dots n} P(C_k | \mathbf{x}_j, \theta_k^g)}$$

Eq. (3)

Il nuovo centroide μ_k è una media pesata di tutti i pattern $\mathbf{x}_j$. Il peso assegnato a $\mathbf{x}_j$ è il suo grado di appartenenza al cluster C_k . I pattern con alta probabilità (vicini al centro) contribuiscono di più.

$$\Sigma_k^{g+1} = \frac{\sum_{j=1 \dots n} \overbrace{P(C_k | \mathbf{x}_j, \theta_k^g)}^{\text{Peso}} (\mathbf{x}_j - \mu_k^{g+1}) (\mathbf{x}_j - \mu_k^{g+1})^t}{\sum_{j=1 \dots n} P(C_k | \mathbf{x}_j, \theta_k^g)}$$

Eq. (4)

Il numeratore calcola la dispersione attesa dei punti attorno al nuovo centroide μ_k .

La divisione per il denominatore assicura che il risultato sia la covarianza media per punto, e non semplicemente la somma totale della dispersione, permettendo alla matrice Σ_k di descrivere correttamente la forma e l'orientamento del cluster.

Intuitivamente, trattando le componenti della mixture come cluster:

- Eq (1), stessa della slide precedente: indica la probabilità di appartenenza di $\mathbf{x}_j$ al cluster C_k . Calcolata sfruttando inversione di Bayes. Ricorda: $p(\cdot | \theta_k)$ è una multinormale con parametri θ_k .
- Eq (2). Calcola il «peso» del cluster C_k in base alla somma delle appartenenze a C_k di tutti i pattern del training set.
- Eq (3) e (4). Calcolano media e matrice di covarianza del cluster C_k pesando i pattern rispetto al loro grado di appartenenza a C_k .

[1] J. A. Bilmes, "A Gentle Tutorial of the EM Algorithm and its Application for Gaussian Mixture and Hidden Markov Models", International Computer Science Institute, 1998.

L'EM richiede l'inizializzazione di tutti i parametri che definiscono le s distribuzioni Gaussiane:

- Pesì (α_k): La probabilità a priori di ciascun cluster (devono sommare a 1).
- Medie (μ_k): La posizione centrale del cluster (il centroide).
- Matrici di Covarianza (Σ_k): La forma e l'orientamento del cluster.

EM: algoritmo

Il processo iterativo dell'EM, che, sebbene sia basato su principi statistici molto più complessi del K-means, ha una struttura a ciclo simile

Inizializza n, s n = numero di pattern
 s = numero di cluster

Inizializza $g = 0, \alpha_k^0, \theta_k^0 = \{\mu_k^0, \Sigma_k^0\}, k = 1 \dots s$

do {

1 **Expectation:** calcola

↪ Calcola la Probabilità di appartenenza di ciascun pattern per ogni cluster

$$P(C_k | \mathbf{x}_j, \theta^g), \quad k = 1 \dots s, j = 1 \dots n \quad // \text{Eq. (1)}$$

2 **Maximization:** calcola

→ Aggiorna i parametri per massimizzare la verosimiglianza. Usando le probabilità di appartenenza come pesi, si aggiornano i pesi (α_k), i centroidi (μ_k), e le matrici di covarianza (Σ_k) per trovare i valori che meglio si adattano alle probabilità attese (calcolate da Eq 1).

$$\alpha_k^{g+1}, \mu_k^{g+1}, \Sigma_k^{g+1}, \quad k = 1 \dots s \quad // \text{Eq. (2) (3) (4)}$$

$$g = g + 1$$

} while (parametri α_k e θ_k non stabili & $g < \max$)

Il ciclo continua finché i parametri (i vettori alpha e theta) non cambiano più significativamente o si raggiunge un numero massimo di iterazioni.

L'algoritmo è simile a fuzzy k-means, ma in questo caso, oltre ai centroidi (qui i vettori medi μ_k^{g+1}), abbiamo maggiori gradi di libertà nella forma dei cluster che possono essere ellissoidali.

↪ La stima di Σ_k permette ai cluster di assumere una forma ellissoidale arbitraria (cluster allungati e inclinati).

Attenzione all'implementazione:

- Le matrici di covarianza non possono essere generate completamente random (è necessario siano simmetriche e definite positive). **Consiglio:** partire con matrici diagonali.
- Le probabilità di appartenenza possono assumere valori molto piccoli e causare problemi numerici. **Consiglio:** usare costanti moltiplicative se necessario e normalizzare alla fine.



in spazi ad alta dimensionalità, specialmente se $\mathbf{x}_j$ è lontano da μ_k .
Moltiplicare o sommare questi numeri piccoli può causare underflow numerico.

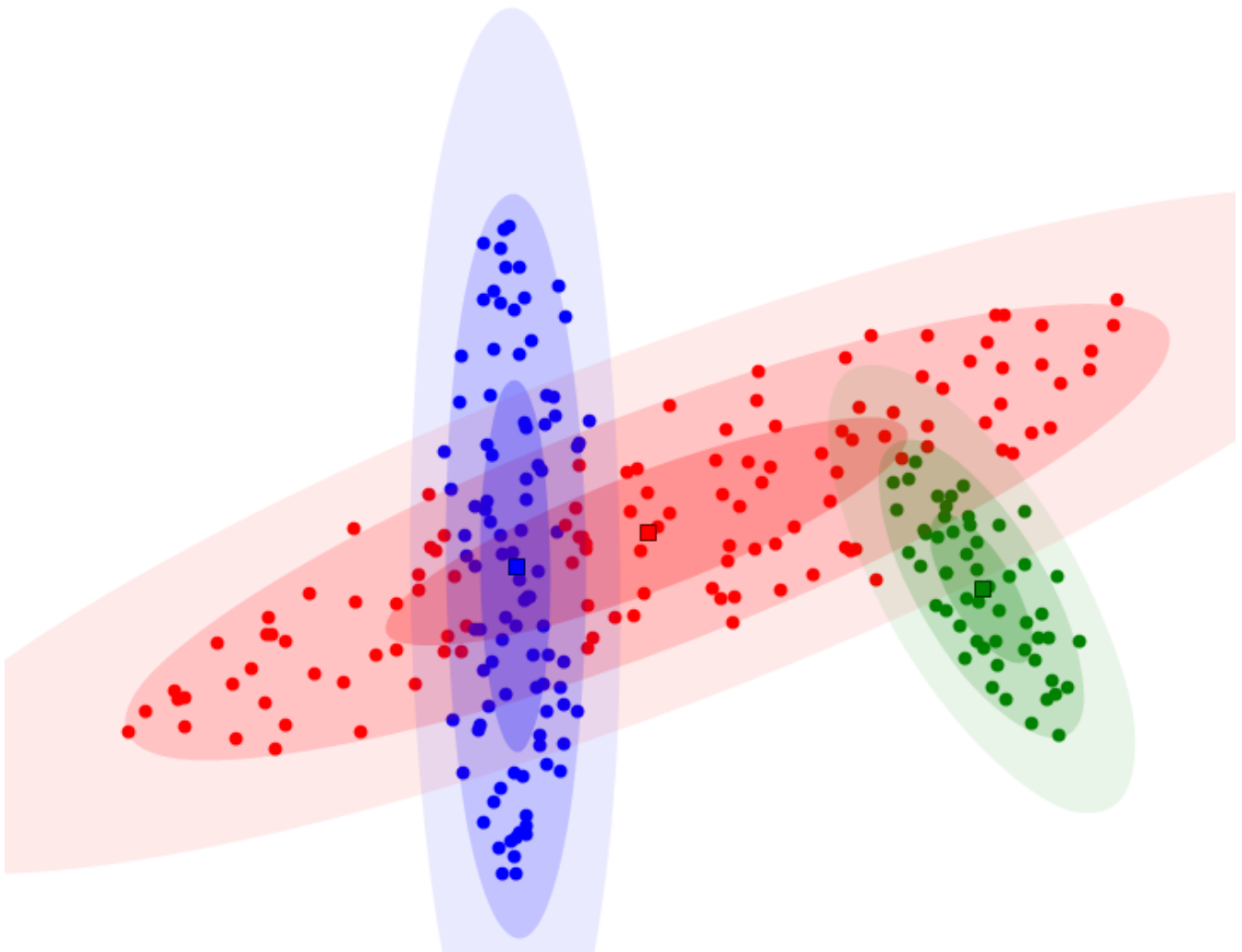
⇒ EM è sensibile all'inizializzazione (dipende da s e n):

- ripetendo più volte l'esecuzione di EM con s diversi, prendo la soluzione migliore che minimizza l'inverso del log likelihood (minimizzare questo è uguale a massimizzare il log del likelihood)
- lavora bene su cluster ellissoidali, sovrapposti, allungati (anche su cluster più complessi: elica, circolari)

Esempio

3 classi – 40 iterazioni circa
(generato con BioLab)

A differenza di K-means e fuzzy K-means,
EM è in grado di identificare nuvole di punti con
forme ellissoidali



Altro esempio

11 classi – 200 iterazioni circa
(generato con BioLab)

